AIN: A Native Language for More Efficient AI

Testing whether a learned native language can reduce AI programming work

Izik Bachar · Research draft 9 · 6 November 2026

It started with a beep

I have spent more than 25 years developing software. The idea came from watching R2D2 beep and plips to other machines and made me realize that communication between machine doesn't need to be set of words a beep can be an entire sentence.

The research question is whether an AI trained to know a computational language could use it to create correct software with less total work. I call the proposed language AIN, for AI Native Language. Its established grammar and operations would be learned before a task, as programming languages are learned during a coding model's training. The model would generate and change AIN directly. Humans could inspect readable projections when needed.

This is a joint question about language structure, learned vocabulary, training and execution. Printing numbers in place of words is only one control condition. A model unfamiliar with AIN and supplied with a manual is also a different condition from a model that already knows it.

My longer-term proposal is AICC, the AI Computational Commons: a governed way to discover, bind and reuse established functionality. AIN concerns how new computation is authored. AICC concerns how much new computation needs to be authored at all. Their effects could interact, but each requires evidence against a competent conventional alternative.

Three claims, with separate measurements

Hypothesis	Possible benefit	What would demonstrate it
N1: native authoring and compilation	Less work to produce a correct runnable program	Matched learned languages; whole generation/repair trajectories; separately measured toolchain time
N2: architecture-aligned representation	Fewer invalid choices, inference steps or repeated work	Correctness-compute comparisons with tokenization, constraints and auxiliary machinery controlled
N3: AI implementations and AICC reuse	Better systems execution or lower repeated development cost	Compiler/graph/kernel comparisons and a separate lifecycle reuse experiment

The article's goal is to investigate these claims, not presume their outcome. Success means a practical reduction in resources for correct unseen work. AIN could fail to beat a well-designed readable IR, or save output positions while losing enough correctness to become more expensive overall.

What native knowledge changes

A native condition should not be charged for a language primer that it never receives. The vocabulary and fixed operational rules are learned during training. Training, tokenizer construction and maintaining that knowledge still cost work; they belong in setup and lifecycle accounting.

New project requirements, data schemas, unfamiliar capability interfaces and private rules can still require context. Knowing a language does not reveal the behavior of every newly published function. AICC discovery and authorization therefore remain possible costs even when AIN itself is already familiar.

This distinction matters to interpreting the current experiments. Claude's operator-graph program uses 298 tokens in both AIN and readable IR under `o200k_base`. Adding their supplied codebooks gives 666 and 580 respectively. That spelling result is useful and was independently recounted, but it does not test a trained native vocabulary or a model generating without a grammar primer. It also does not measure billing. The Python entry in that table includes more operator logic and is explicitly not a matched comparison.

Writing, compiling and running are different costs

A language could save work at several points. A model might emit fewer positions, make fewer invalid choices or need fewer repair attempts. A deterministic compiler might parse or check a structured form efficiently. The resulting program might also permit useful optimization. These outcomes should be measured separately.

LLVM, MLIR and WebAssembly already establish substantial precedent for executable machine-oriented representations and readable views. AIN's contribution must therefore be more specific than having opcodes or a compact serialization. The proposed difference is a representation learned and used by AI, with a measured benefit over equally capable source and IR controls. (MLIR, WebAssembly)

For compilation, compare equivalent programs lowered to the same target with the same optimization opportunity. Report parsing, type/effect checking, lowering, backend work and the complete time to a correct runnable artifact. A faster frontend might contribute little when model inference or optimization dominates. An interpreter benchmark is not an optimizing-compiler comparison.

What I mean by less "traveling"

The intuition is that AI should spend less effort reaching a correct computational structure. Scientific measurements need to replace the metaphor: decoding and latent steps, repeated context processing, invalid candidates, repairs, actual hardware time and memory.

In a normal fixed-depth dense Transformer, a native symbol still uses the model's layer stack per forward step. Machine-like spelling alone does not establish a shorter neural path. A useful gain could instead come from fewer necessary steps or more reliable choices. This is an architectural inference, not a result from our AIN prototype. (Transformer paper)

The literature supports investigating structure. TyFlow trains models on type/synthesis decisions and reports improved selected code outcomes, although its representation is combined with pruning and

context machinery. Conventional languages can also benefit from type-constrained decoding. Both observations make equally equipped controls essential. (TyFlow, Type-Constrained Code Generation)

Fewer steps are not an unconditional objective. Pause-token training shows that additional computation can improve answers. Coconut explores trained latent reasoning with task-dependent accuracy and efficiency. These findings motivate comparing correctness and compute jointly rather than rewarding the shortest trace. (Pause Tokens, Coconut)

Closely related evidence

Native computational learning is technically possible in restricted settings. Cadmus trains on an integer virtual machine with fixed instruction tokens and no natural-language training. Its narrow program-completion setting does not establish general programming efficiency, but it is a direct precedent for the kind of language fluency this proposal requires. (Cadmus)

IRCoder learns source-compiler-IR alignment during continued pretraining, with useful code-task gains. This supports learning such representations in weights; it does not measure an AIN authoring-cost advantage. (IRCoder)

Learned compression provides both encouragement and caution. zip2zip trains models to use compressed text units, with token/throughput gains and quality tradeoffs. Broader tokenization studies show that minimum sequence length and greater compression are not sufficient objectives for model quality or compute efficiency. None supplies an AIN saving percentage. (zip2zip, Tokenization Is More Than Compression, Compute Optimal Tokenization)

The broad principles have prior art. The open contribution is whether this AIN design improves correct, complex program construction and maintenance under a fair resource comparison. Existing evidence makes a bounded experiment reasonable; it does not make the desired result inevitable.

What our project has tested

The earlier processing research contains 135 complete worker records from the original study and two submitted remeasurements. They repeat three public million-event datasets. Optimized ordinary code improves runtime and memory, and the same implementations work through the AICC binding prototype. These are algorithm and systems results, not native-language inference savings.

The study's 867-parameter neural network selects among supplied processing plans. It did not consistently beat a strong simple policy. It does not learn AIN. Detailed timings, granularity experiments and limitations remain in the processing synthesis and unchanged Draft 8.

The newer Codex AIN v0 implementation exposes branches and loops over shared host services. Its recorded engineering suite has 69 passing methods; 27 20,000-event processing workers agree on outputs and committed state. Six fresh same-host agent contexts each submit one repair or requested optimization, and all pass their judges. These are small public development cells, not a blinded model-authoring comparison. The model weights were not trained on AIN and actual provider identity/billing was unavailable.

In that instrumented Python implementation, numeric AIN execution medians are 22.5-46.5 percent higher than direct Python across the workloads. This adverse prototype-runtime result is retained. It measures a different cost from native model generation. (First execution report)

Claude separately implements the earlier operator sketch, with static stream typing, semantic decisions and mutation tests. Its published regression outcomes reproduce. Review also exposes shared gaps: records after END are ignored on the first invocation, malformed restored state can produce negative counts, and failed test gates still return a successful process status. Those findings show why agreeing implementations and many passing cases do not certify a judge. (Claude source, review evidence)

We have built useful executable foundations. We have not trained and evaluated an AIN-native model. Earlier contributor opinions calling the language validated or rejected cannot substitute for that missing experiment.

A substantial program, rather than a hidden solution

The intended task remains a transactional streaming workflow: out-of-order events, finite-horizon deduplication, anchored groups, Unicode normalization, private policy, bounded asynchronous work, cancellation and atomic recovery. A low-severity first event still reserves its ID; a late event does not. A lost acknowledgment cannot duplicate publication. An optimization must preserve those interactions and the configured resource limits.

For an authoring study, agents need enough language to implement or change the logic being judged. A fixed graph that merely wires prewritten whole-task operators is a composition test. The next core should expose the relevant state/container operations, conditions, loops and effects. All representations must receive the same legitimate normalization, private-policy and sink services. The complex task contract supplies behavioral obligations, not proof that every implementation already satisfies them.

AI implemented in AIN and the AICC layer

There is a further systems question: could AI software itself be implemented more efficiently in AIN? That requires distinguishing its source language from its learned weights and executed computation graph. OpenXLA already optimizes and lowers neural graphs to hardware code. If a source rewrite produces the same graph and kernels, no new execution saving has been identified. AIN would need to enable better computation, scheduling, memory behavior or development work. (OpenXLA architecture)

AICC offers a separate opportunity to avoid repeated implementation and compatibility work. It needs precise behavior, version/artifact identity, private domains and host-enforced authority. A short handle is not encryption, a complete contract or proof of safe substitution.

Ordinary libraries already permit calls without reading their bodies. DreamCoder provides prior work on learned reusable abstractions, and Unison identifies code by content rather than its human name. AICC therefore needs a demonstrated governance and lifecycle contribution beyond ordinary reuse. (DreamCoder, Unison)

The eventual comparison should cross trained representation with ordinary versus AICC binding while holding implementations and semantic evidence constant. It can then estimate language effects, reuse effects and their interaction. Upfront training, discovery, verification, binding and maintenance costs must be repaid by actual recurring gains; a fixed break-even count cannot be assumed.

The experiment that should decide the next investment

The main next gate is matched native learning. First repair and freeze the semantic core, data generator and independent judge. Then train small matched models from scratch, or explicitly label a study as matched adaptation of a pretrained base. Evaluation supplies task information without reteaching the fixed language. A trainable checkpoint and suitable vocabulary/embedding interface are needed; prompting an assistant is not parameter training.

Use equivalent executable programs and hold out structural templates, compositions and task families as well as event seeds. Compare competent conventional source, readable typed IR, mechanical numeric spelling, and any structurally distinct AIN candidate. Merge forms that coincide. Give controls the same semantic-token, constraint, library, diagnostic and optimization opportunities.

Report both equal semantic exposure and equal training-compute conditions where feasible. Equal raw tokens need not contain equal semantic training information. Record vocabulary/output-head cost and auxiliary encoders as well as the main model. Select checkpoints using validation only; freeze practical margins, correctness gates, resource limits and uncertainty methods before final tasks.

The main outcome is total resources from all assigned attempts divided by correct completed task chains, reported with success rate. Include failed attempts, repairs, tool work and human intervention. Show compilation time, emitted-program runtime and training amortization separately. Cross-tokenizer unit counts are descriptive and do not automatically correspond to provider billing.

If AIN fails correctness or a useful advantage against the strongest control within the declared budget, pause scaling and publish that result. If a readable IR matches AIN, attribute any gain to the actual shared mechanism. A small study can be inconclusive; it should not become an unlimited search for a favorable headline. The draft protocol records the remaining configuration and budget decisions. No paid training run is begun by this article.

What this article can claim

There are two levels of result. If both AIN and learned readable IR beat competent conventional source, that supports the broader value of a learned computational representation. A tie between them would limit claims specific to AIN's distinct design, rather than erase the shared gain. The successful mechanism could be adopted without insisting that opaque spelling is its cause.

The native-language hypothesis is possible and has close precedents. Our current project supplies an executable foundation and informative negative controls. Its specific AI-usage and lifecycle benefits remain unmeasured. The useful next contribution is a controlled answer about correct new computation, with an explicit reason to continue or stop.

Credits and evidence scope

Concept, architecture, research direction and article approval: **Izik Bachar**. Earlier drafting and exploratory review involved ChatGPT/OpenAI, Claude/Anthropic, Gemini/Google and Grok/xAI as recorded in the preserved material. Draft 9, repository/literature synthesis and editorial assistance: OpenAI Codex and its collaboration agents under Izik's direction. Same-session agents are not counted as independent provider replications. Submitted provider/model identities remain self-reported unless separately attested.

This revision preserves the author's requested beep opening and the measured artifacts. It supersedes earlier editorial priorities for this article; it does not rewrite historical results. The full feasibility review and primary-source map distinguishes paper reports, official design sources, project measurements and proposed work. External model experiments were reviewed, not independently reproduced here. The original full ChatGPT conversation and editable original Draft 4 remain unavailable; no whole verbatim chat transfer is claimed.